In this section, we connect Dynamic Programming (DP) with Reinforcement Learning (RL) methods through a concrete navigation example. Both methods attempt to find optimal policies in Markov Decision Processes (MDPs). An MDP is defined as a tuple (S, A, P, R, γ), where S denotes the state space, A the action or choice space, $P(s'|s, a)$ the transition probability function, $R(s, a, s')$ the reward or utility function, and $\gamma \in [0, 1]$ the discount factor. Policies π are conditional probabilities functions that tell us the probability of taking actions in states.

Following Sutton and Barto's formulation, Generalized Policy Iteration (GPI) refers to the interaction between policy evaluation and policy improvement processes. Policy evaluation makes the value of being in states, i.e. the an estimate of the expected discounted returns, when following the current policy. Policy improvement finds the best policy given a valuation of the states. When both processes stabilize, the resulting policy and value function is optimal.

0.0.1 Dynamic Programming

First, we look at two DP techniques. The solution is stored in value and policy tables, initialized at 0. which sweep through, updating all states at each iteration (full-width). They both look ahead one step (depth=1), and use the previous guess of the value function to update (bootstrapping). The learning rate is 1, i.e. it is a full update. At each iteration, Value Iteration (VI) computes the optimal value function through the Bellman optimality equation:

$$\text{Value Iteration:} \quad \forall s, V(s) \leftarrow \max_a \sum_{s'} P(s'|s, a)[R(s, a, s') + \gamma V(s')] \tag{1}$$

The max operator delivers policy improvement, while the assignment delivers value evaluation. By requiring transition probabilities and potential rewards across all actions and future states, VI is inherently model-based. Theoretically, VI converges to the optimal value function at a linear rate, with error decaying geometrically with ratio γ **?**. For any finite MDP with $\gamma < 1$, VI guarantees convergence to V^* from any initial V_0 **?**.

Policy Iteration (PI) Alternates between policy evaluation and improvement:

$$\text{Policy Iteration:} \quad V^{\pi}(s) = \sum_{s'} P(s'|s, \pi(s))[R(s, \pi(s), s') + \gamma V^{\pi}(s')] \tag{2}$$

$$\pi'(s) = \underset{a}{argmax} \sum_{s'} P(s'|s, a)[R(s, a, s') + \gamma V^{\pi}(s')] \tag{3}$$

This method separates policy evaluation and policy improvement, and begins with a random policy, and recursively finds the value of states for it. PI converges to an optimal policy in a finite number of iterations, with each policy improvement yielding a strictly better policy until optimality is reached **?**.

0.0.2 Reinforcement Learning

We now turn to Reinforcement Learning (RL) methods, which do not require knowledge of P and R functions but only require access to transitions and rewards only at those particular actions and states (i.e. a simulator). The first two are temporal-difference methods that learn value tables. They begin from an initial state and explore their environment one state at a time (width=1) and look ahead one step (depth=1), using their previous estimate of the value function (bootstrapping). The actions are not selected greedily, but ϵ-greedily which allows for randomness and exploration of the state space. The updates also happen incrementally with the learning rate $\alpha < 1$.

Q-learning is an off-policy algorithm that updates action-values using individual sampled experiences, following the update rule:

$$\text{Q-learning:} \quad Q(s, a) \leftarrow Q(s, a) + \alpha[R + \gamma \max_{a'} Q(s', a') - Q(s, a)] \tag{4}$$

Q-Learning as Stochastic Value Iteration We can formally demonstrate that Q-learning is a stochastic approximation of value iteration. Let us define the optimal Q-factor, $Q^*(i, u)$, as the expected discounted cost of taking action u in state i and following the optimal policy thereafter. The Bellman optimality equation for Q-factors is:

$$Q^*(i, u) = \sum_j p_{ij}(u) \left(g(i, u, j) + \alpha \min_v Q^*(j, v) \right) \tag{5}$$

This equation suggests a value iteration algorithm for Q-factors:

$$Q_{k+1}(i,u) = \sum_j p_{ij}(u) \left(g(i,u,j) + \alpha \min_v Q_k(j,v) \right) \tag{6}$$

In a model-free setting, the transition probabilities $p_{ij}(u)$ are unknown, so the expectation (the summation over j) cannot be computed. Q-learning approximates this expectation using a single sample (s', r) obtained from taking action u in state i. The Q-learning update rule can be written as:

$$Q_{k+1}(i,u) = (1-\gamma_k)Q_k(i,u) + \gamma_k \left(r + \alpha \min_v Q_k(s',v) \right) \tag{7}$$

where γ_k is the learning rate. This is a stochastic approximation of the value iteration update in Eq. **??**. As shown in **?**, this can be analyzed as a stochastic iterative algorithm. The update can be expressed as $Q_{k+1} = (1-\gamma_k)Q_k + \gamma_k(HQ_k + w_k)$, where H is the Bellman operator and w_k is a zero-mean noise term. The convergence of Q-learning is guaranteed if the operator H is a contraction mapping, which is the case for discounted problems.

At each step, the agent observes a tuple (s, a, s', r), chooses actions using an ε-greedy policy (selecting random actions with probability ε and the best-known actions with probability $1-\varepsilon$), and updates its value estimates. Over time, ε typically decreases, shifting from exploration to exploitation. Under conditions of infinite exploration, diminishing learning rates, and greedy-in-the-limit behavior, Q-learning converges to the optimal action-value function Q^* with probability 1 (**?**).

SARSA for State-Action-Reward-State-Action updates action-values using the action selected by the current policy for the next state:

$$\text{SARSA:} \quad Q(s,a) \leftarrow Q(s,a) + \alpha[R + \gamma Q(s',a') - Q(s,a)] \tag{8}$$

Unlike Q-learning, SARSA computes the Bellman error based on actual behavior (next action chosen by the policy) and not greedy-behavior, making it on-policy i.e. the same policy that generated the samples is being updated. It similarly uses ε-greedy exploration to generate samples (s, a, s', r, a'). SARSA converges to the optimal Q^* with probability 1 when coupled with a policy that becomes greedy in the limit, provided every state-action

pair is visited infinitely often **?**.

Monte Carlo methods are on-policy methods that learn from complete episodes, updating action-values based on observed returns:

$$\text{Monte Carlo:} \quad Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[G_t - Q(s_t, a_t)] \tag{9}$$

They begin from random policies, which are traced out till termination (reaching the goal or being truncated), and then updates are made to each state-action visited. Here, the return $G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}$ represents cumulative discounted rewards from time t to the episode's end. This does use bootstrapping: it does not estimate value using existing estimates like $V(s')$ or $Q(s', a')$. Instead, it waits for the full return to be observed before making updates, making the updates unbiased. For episodic tasks with exploring starts, Monte Carlo methods converge to the optimal policy as the number of episodes approaches infinity **?**.

Lastly, we touch on policy-gradient methods. REINFORCE directly optimizes the policy parameters θ based on observed episode returns G, following the gradient direction that increases the probability of actions leading to higher rewards. The policy parameterized by, typically, a neural network:

$$\text{REINFORCE:} \quad \theta \leftarrow \theta + \alpha \nabla_\theta \log \pi_\theta(a|s) G \tag{10}$$

One can interpret the REINFORCE as trying to generate data from a policy and re-estimating the policy parameters using maximum likelihood via gradient ascent, but with increased weight when the trajectory returns larger returns. This step pulls the policy towards generating larger returns. REINFORCE converges to a stationary point of the expected return function with probability 1 (**?**). In tabular settings with suitable exploration, it achieves global optimality at a geometric rate **?**.

0.1 Example 1: Grid Navigation

To illustrate the connection between DP and RL methods, we consider a 5×5 grid world navigation problem forming a finite MDP where states are coordinates (r, c) with $0 \leq r, c < 5$; actions are directional movements {L, R, U, D, S}[1]; transitions follow deter-

[1] L=Left, R=Right, U=Up, D=Down, S=Stay

ministic movement rules with boundary constraints; rewards include +10 at the terminal state $(4, 4)$, smaller intermediate rewards throughout the grid, and a movement penalty of -0.1; and the discount factor γ equals 0.95. Table **??** presents solutions from all tabular methods.

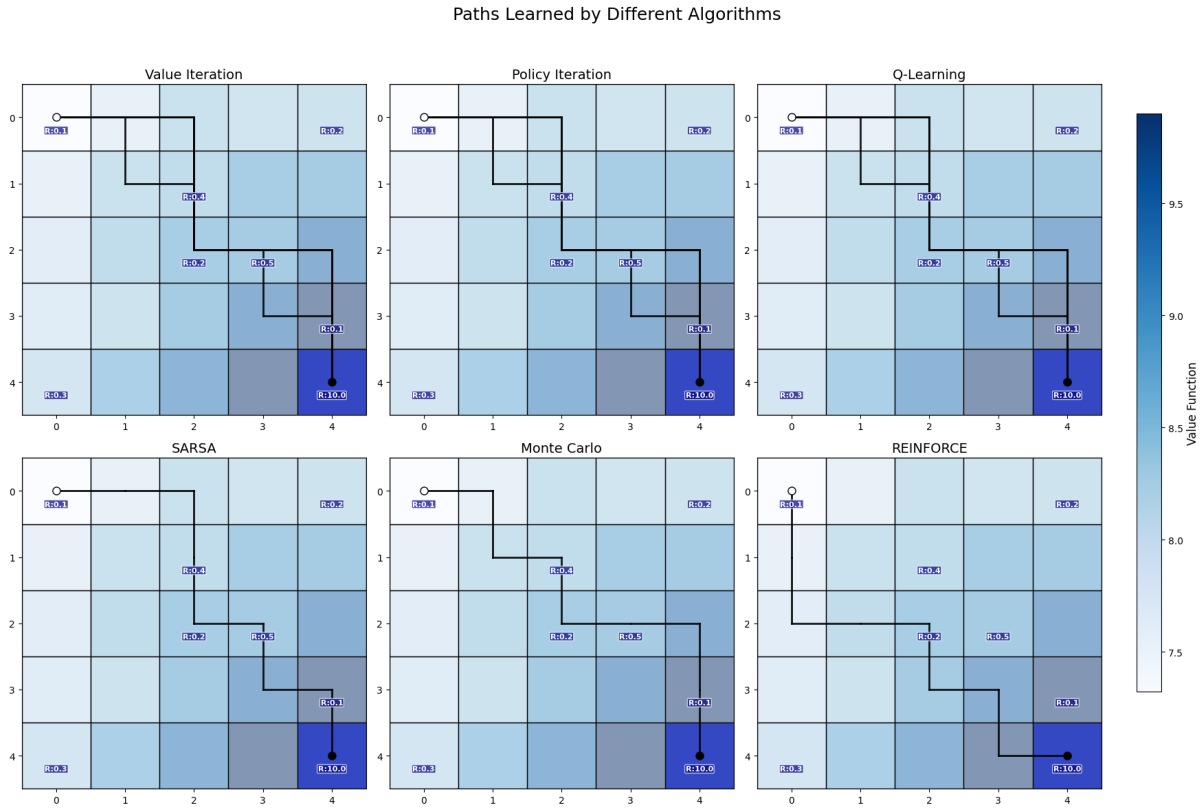


Figure 1: Paths found by each algorithm.

Table 1: Performance comparison of DP and RL methods on the grid world problem[2]

Method	**MSE***	**Policy Match***	**Time (s)**	**NumIter**	**Episodes***	**Return***
VI	0.00	1.00	0.00	9	8.00	10.40
PI	0.00	1.00	0.01	7	8.00	10.40
Q-Learning	0.00	1.00	0.04	1211	8.00	10.40
SARSA	1.57	0.92	0.08	5000	8.00	10.40
Monte Carlo	0.12	0.88	0.09	5000	8.00	10.40
REINFORCE	72.94	0.75	12.97	5008	8.00	9.40

VI and PI converge exactly and rapidly to optimal solutions with perfect policy matches, leveraging their complete knowledge of the MDP. Q-learning achieves perfect policy matching despite lacking prior model knowledge, though requiring more iterations to converge. SARSA and Monte Carlo find valid paths to the goal but do not identify all optimal actions, instead converging to a single path with reasonable value approximation. As shown in Figure **??**, these RL methods favor consistent exploitation once a workable solution is found. REINFORCE shows the most variable performance, achieving only 75% policy matching and suboptimal rewards, highlighting the sensitivity of policy gradient methods to hyperparameter tuning and neural network initialization. This example illustrates that while model-free RL methods can solve tasks without prior knowledge of transition and reward functions, they typically require more samples and tuning and may not identify all optimal actions that DP methods discover.

0.2 Deep Reinforcement Learning

Deep Q-Network (DQN) extends Q-learning by approximating the action-value function with a deep neural network $Q(s, a; \theta)$. Instead of storing values in a table, DQN uses a parameterized model trained on transition samples (s, a, r, s') drawn from a replay buffer. Its objective is to minimize the temporal-difference (TD) error:

$$L(\theta) \;=\; \mathbb{E}_{(s,a,r,s')}\Big[\Big(r \;+\; \gamma \max_{a'} Q(s', a'; \theta^-) \;-\; Q(s, a; \theta)\Big)^2\Big].$$

Here, θ^- denotes the parameters of a separate target network, periodically synchronized to prevent instability from moving targets. By replaying past transitions and using a target network, DQN decouples correlated updates and stabilizes learning, enabling it to scale to high-dimensional state spaces. Under proper conditions including Lipschitz-continuous Q-function and sufficient exploration, DQN converges to an approximate Bell-

man fixed point **?**. With experience replay and target networks breaking the "deadly triad" instability, DQN can achieve geometric convergence to Q^* when the network class can represent it **?**.

0.3 Example 2: Capital Replacement

We study a maintenance optimization problem with N parallel engines formulated as a Markov Decision Process. Each engine i has mileage state $m_i \in \{0, 1, 2, 3, 4, 5\}$, with system state $s = (m_1, ..., m_N)$. The action space $A(s)$ consists of all subsets of engines to replace, constrained by capacity $C = 3$. The cost function is:

$$c(s, a) = \alpha \sum_{i=1}^{N} \mathbf{1}\{m_i > 0\} + \beta |a| \tag{11}$$

where $\alpha = 1.0$ is the cost of aged engines, $\beta = 5.0$ is the replacement cost, and $|a|$ is the number of replaced engines. State transition is deterministic:

$$m_i' = \begin{cases} 0 & \text{if } i \in a \\ \min(5, m_i + 1) & \text{otherwise} \end{cases} \tag{12}$$

The objective is to find policy π^* minimizing expected discounted cost with $\gamma = 0.95$:

$$V^*(s) = \min_{\pi} \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t c(s_t, \pi(s_t)) \right] \tag{13}$$

For the DQN implementation, we map the system state $s = (m_1, \ldots, m_N)$ directly to the neural network input[3], where each mileage value m_i becomes a feature in the input vector. The action space is encoded as indices of a fixed-size output vector, where index j corresponds to the j-th possible subset of engines to replace (e.g., empty set, single engines, pairs, and triples up to capacity $C = 3$).

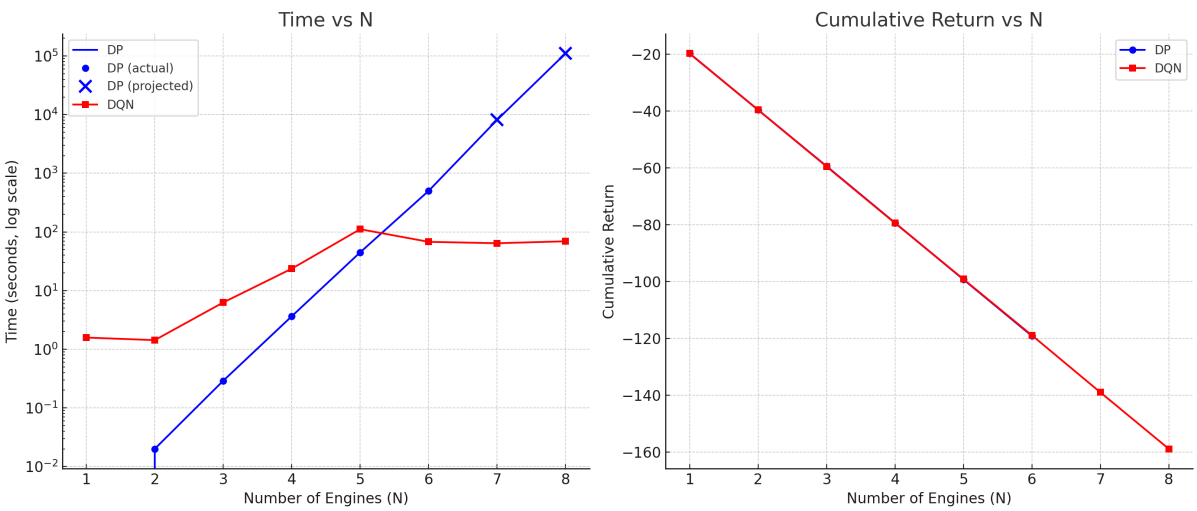
Dynamic Programming (DP) solves this exactly via value iteration, but faces exponential state growth $\mathcal{O}(6^N)$. For each state, DP evaluates all possible actions and selects the one minimizing the Bellman equation until convergence, measured by maximum value

[3]The neural network architecture consists of fully-connected layers (128-64) with ReLU activations, trained using Adam optimizer with learning rate 1e-3 and batch size 64. To ensure stability, we employ ϵ-greedy exploration starting at $\epsilon = 1.0$ and decaying by a factor of 0.995 per episode to a minimum of 0.01, along with a target network updated every 50 episodes.

change between iterations falling below 1e-5. DQN addresses the scalability challenge by approximating the value function and learning from sampled experiences, with convergence measured through the Bellman residual—the average absolute difference between the current Q-value estimate and the target.

N	#States	DP Time	DP Reward	DQN Time	DQN Reward	DQN Bell Res
1	6	0.00s	-19.80	1.58s	-19.83	0.44
2	36	0.02s	-39.64	1.43s	-39.63	0.86
3	216	0.29s	-59.42	6.27s	-59.53	0.79
4	1,296	3.64s	-79.35	23.60s	-79.43	2.13
5	7,776	44.82s	-99.26	111.33s	-99.15	1.82
6	46,656	500.98s	-119.10	67.81s	-118.88	2.79
7	279,936	∼6418.84s	–	64.18s	-138.93	2.58
8	1,679,616	∼78561.36s	–	69.25s	-158.80	2.92

For small instances ($N \leq 3$), dynamic programming (DP) yields exact solutions in under 1 second, while deep Q-networks (DQN) require 20–30 seconds to achieve comparable performance. As the state space grows, DP becomes intractable—solving $N = 6$ (46,656 states) takes over 500 seconds, and projections for $N = 7$ exceed 6,000 seconds. In contrast, DQN scales efficiently and maintains high solution quality, with rewards closely matching those from DP wherever comparison is possible.



0.4 Performance of Reinforcement Learning on Optimization Problems

Multiple empirical studies provide evidence that Deep RL can indeed overcome the curse of dimensionality that limits traditional Dynamic Programming. **?** showed that for per-

ishable inventory control with dynamic pricing, PPO achieved near-optimal solutions with a 75% reduction in computation time compared to DP, with authors explicitly noting that increasing problem dimensionality would render DP impractical while DRL would remain viable. **?** demonstrated that attention-based DRL delivered maintenance schedules with 30.5% lower costs than DP-based heuristics on combinatorial problems, computing in seconds what DP required over an hour to solve. For high-dimensional inventory problems where exact DP was infeasible, **?** showed that A3C matched the performance of the best specialized heuristics and approximate DP methods across three inventory settings. These preliminary results show that while Deep RL requires substantial initial training experiences and hyperparameter tuning, they can effectively approximate optimal policies in high-dimensional state spaces where exact DP becomes computationally prohibitive.

0.5 Theoretical Foundations Connecting DP and RL (2020–2025)

The relationship between Dynamic Programming and Reinforcement Learning has become mathematically precise in recent years, with tight sample complexity bounds, convergence guarantees, and formal equivalence results now available across multiple settings relevant to economics. We survey the key theoretical advances organized by theme.

0.5.1 Convergence and Sample Complexity: Q-Learning Meets Minimax Optimality

The most significant recent theoretical advance establishes **tight sample complexity bounds** that precisely characterize when RL algorithms match DP's efficiency. **?** prove that vanilla Q-learning is strictly suboptimal, requiring $\tilde{O}(SA/(1-\gamma)^5\varepsilon^2)$ samples—worse than the minimax lower bound $\Omega(SA/(1-\gamma)^3\varepsilon^2)$ by a factor of $(1-\gamma)^{-1}$. This gap arises from overestimation bias in bootstrapping, connecting directly to the Bellman optimality equation's maximum operator.

? establish finite-time convergence rates for TD learning with linear function approximation by connecting TD updates to online gradient descent. Their analysis extends to TD(λ) and Q-learning for optimal stopping, providing explicit non-asymptotic bounds under both i.i.d. and Markovian sampling. For average-reward settings, **?** characterize optimal $O(1/k)$ convergence rates for Value Iteration, establishing span-based complexity that matches lower bounds within a factor of 8.

? address the "deadly triad" directly, proving convergence of modified Q-learning with linear function approximation under general assumptions. Their work establishes rigorous bounds connecting the Q-learning fixed point to the DP optimal Q-function—a result particularly relevant for econometric applications using linear approximations.

Additional contributions include **?**, who establish nearly minimax optimal RL for discounted MDPs, and **?**, who provide a rigorous analysis of quantile temporal-difference learning.

0.5.2 Model-Based Methods Provably Dominate in Stationary Environments

A striking theoretical consensus has emerged: **model-based RL (closer to DP) achieves optimal sample complexity** while model-free methods typically incur polynomial overhead. **?** settle a long-standing open problem by proving that model-based algorithms achieve minimax-optimal regret $\tilde{O}(\sqrt{SAH^3K})$ without any burn-in cost—eliminating the warmup requirements that plagued model-free methods.

? prove that the model-based plug-in approach achieves minimax-optimal sample complexity for offline RL with distribution shift, achieving $SC^*/(1-\gamma)^3\varepsilon^2$ where C^* is a concentrability coefficient. Their companion paper (**?**) breaks the S^2A sample size barrier, showing model-based algorithms need only $\tilde{O}(SA/(1-\gamma)^3\varepsilon^2)$ samples.

? provide a novel **circuit complexity explanation** for these gaps: transition and reward functions can be represented by constant-depth polynomial-size circuits, while optimal Q-functions require exponential circuit complexity. This fundamental representation asymmetry explains model-based advantages from first principles.

The one exception is non-stationary environments. **?** prove that model-free methods achieve superior dynamic regret $\tilde{O}(S^{1/3}A^{1/3}\Delta^{1/3}HT^{2/3})$ when dynamics change, since model estimation becomes a liability—an important consideration for econometric applications with structural breaks.

0.5.3 Value Equivalence: When Learned Models Suffice for Planning

The theoretical foundations for combining DP-style planning with learned models have matured considerably. **?** introduce the **value equivalence principle**: two models are value equivalent if they yield identical Bellman operator updates for a given set of policies and value functions. As these sets expand, the class of value-equivalent models contracts

to the true model—but crucially, smaller sets permit useful approximations.

? extend this with **proper value equivalence (PVE)**, proving that PVE models are sufficient for planning even when they differ from ground truth. They construct a loss function for learning PVE models and demonstrate that MuZero minimizes an upper bound of this loss, providing the first rigorous justification for MuZero's empirical success.

? introduce value-targeted regression, proving regret bounds that depend on value function structure rather than state space size. Learning models to minimize value prediction error (rather than maximum likelihood) yields better sample complexity—a decision-aware approach directly relevant to structural estimation where the goal is policy evaluation, not dynamics prediction.

? provide optimally tight bounds for the **simulation lemma**, the foundational result quantifying value-prediction error under model misspecification. Their bound is sub-linear in transition function misspecification, improving error propagation analysis for all model-based methods. Additional contributions include **?** on deciding when to trust learned models, and **?** on gap-dependent sample complexity for planning.

0.5.4 Neural Network Approximation: Bridging Approximate DP and Deep RL

The theoretical gap between classical approximate DP and deep RL has narrowed substantially. **?** provide the first rigorous analysis of DQN, establishing that neural Fitted Q-Iteration converges geometrically to the optimal Q-function up to intrinsic statistical error induced by **ReLU network approximation bias** and finite sample size. They provide theoretical justification for experience replay and target networks.

? prove that target networks can break the deadly triad by proposing Polyak-averaging updates with projections that converge to regularized TD fixed points. **?** establish **order-optimal sample complexity** $\tilde{O}(1/\varepsilon^2)$ for Fitted Q-Iteration with two-layer ReLU networks without requiring linear or low-rank MDP structure.

? establish the first provably efficient RL algorithm with general nonlinear value function approximation, achieving regret $\tilde{O}(\text{poly}(dH)\sqrt{T})$ where d depends on **eluder dimension** and log-covering numbers—a complexity measure that extends beyond linear settings while remaining tractable.

On estimation bias, **?** provide rigorous analysis of underestimation bias in Double Q-learning, while **?** show how Maxmin Q-learning can systematically control bias from

positive to negative. These results connect to DP's Bellman operator properties. **?** contribute further with analysis of regularized Q-learning.

0.5.5 Generalized Policy Iteration Achieves Linear Convergence

The interaction between policy evaluation and policy improvement—the core of both DP and RL—is now understood through the lens of **gradient domination properties**. **?** establish global convergence for policy gradient using a Polyak-like gradient domination property, introducing the distribution mismatch coefficient framework that governs GPI convergence rates.

? develops weak gradient-mapping dominance theory, proving $O(1/k)$ convergence for projected policy gradient and showing policy mirror descent achieves linear convergence with geometrically increasing step sizes—without entropy regularization. Crucially, as step sizes approach infinity, policy mirror descent reduces to classical Policy Iteration, establishing a continuum between gradient-based and value-based methods.

? prove non-asymptotic linear convergence (plus local quadratic convergence) for entropy-regularized Natural Policy Gradient, with iteration complexity $O((1/(1-\gamma))\log(1/\varepsilon))$. **?** provide the first non-asymptotic analysis of **two-timescale actor-critic** with single Markovian sample paths, establishing $O(\varepsilon^{-2.5})$ sample complexity.

For deep networks, **?** extend actor-critic convergence from shallow to arbitrary-depth networks, showing convergence to neighborhoods where average squared gradients scale as $\tilde{O}(1/\sqrt{m}) + O(\varepsilon)$. **?** analyze NPG as gradient flows in Hessian geometries, proving local quadratic convergence when step sizes equal inverse regularization strength. Additional foundational work includes **?** on regularized MDPs.

0.5.6 Synthesis: Implications for Economics

The theoretical literature reveals several key insights for integrating RL into economic methods.

Model-based approaches align with economic estimation. The proven optimality of model-based RL (**??**) supports econometric approaches that estimate structural primitives before solving for optimal policies. The $(1-\gamma)^{-1}$ factor advantage over model-free methods becomes substantial for patient agents with high discount factors—precisely the

long-horizon settings common in lifecycle models.

Value equivalence relaxes full-model requirements. The value equivalence principle (**?**) suggests that structural models need not perfectly replicate all dynamics—only those relevant for policy evaluation. This provides theoretical grounding for parsimonious structural specifications that capture decision-relevant features without full realism.

Function approximation bounds inform practical choices. The eluder dimension framework (**?**) and neural network convergence results (**??**) provide guidance on approximation architectures. Linear function approximation retains strong guarantees (**?**), while neural approximations require careful bias control.

GPI theory unifies estimation and optimization. The gradient domination framework establishes that policy evaluation and improvement can be interleaved without losing convergence guarantees—supporting iterative estimation-optimization procedures common in structural work. The continuum between policy gradient and policy iteration (**?**) offers flexibility in algorithm design.

Distribution mismatch matters for counterfactuals. The distribution mismatch coefficient (**?**) and concentrability coefficients (**?**) quantify how off-policy evaluation degrades with distribution shift—directly relevant for counterfactual policy analysis where target policies differ substantially from observed behavior.